Ders 21 Genişletilmiş Notu: Vize Sorularının Çözümü, Graf Temelleri ve Asgari Tarama Ağacı

Ders 21 Genişletilmiş Notu: Vize Sorularının Çözümü, Graf Temelleri ve Asgari Tarama Ağacı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Vize Sorularının Üzerinden Geçilmesi
- 2. Graf Nedir?
- 3. Grafların Kullanım Alanları
- 4. Düğüm, Kenar ve Grafın Saklanması
- 5. Self-Loop, Paralel Kenar, Multigraf ve Basit Graf
- 6. Yol, Çevrim ve Bağlantılı Bileşenler
- 7. Graf Türleri: Yönlü, Yönsüz, Ağırlıklı, Ağırlıksız
- 8. Çevrimsiz Graflar ve Ağaçlar
- 9. Graf Yoğunluğu: Seyrek ve Yoğun Graflar
- 10. İki Parçalı Graflar
- 11. Graf Gösterimleri
- 12. Asgari Tarama Ağacı Problemi

1. Vize Sorularının Üzerinden Geçilmesi

Hoca derse yeni konuya geçmeden önce ara sınavın (vize) birinci kısmındaki soruları tek tek çözerek başladı. Soruların sırası öğrenciden öğrenciye rastgele değiştiği için hoca kendi yazdığı sırayı kullandı. Soruların tam metni ve derste verilen çözümleri, aynı sırayla, bu notun sonundaki “Örnek Sorular” bölümündedir. Bu bölümde yalnız hocanın sorular üzerinden verdiği genel dersler ve sorulan sorular yer alır.

Hocanın genel mesajı, soruların zor olmadığı ama ayrıntıların dikkatle okunması gerektiğiydi. “Şeytan ayrıntıda gizlidir” dedi. Hatalı cevapların çoğu, sorudaki küçük bir ifadenin gözden kaçmasından kaynaklanıyordu:

- “En kötü durum” ifadesi (BST’ye ekleme, kuyruğa ekleme).
- “Tamamı aynı sayıdan oluşan” dizi (sıralama algoritmalarının karmaşıklığı).
- “İlk büyük değer” (BST’de bir düğümü sıralamada izleyen değer).
- “Son düğüm olmadığı bilinen” düğüm (bağlı listede `x->next->next`).

> **Hocanın vurgusu:** Soruda ne soruluyorsa o cevaplanmalıdır. Fazladan sayı yazmak, yanlış ek bilgi vermek ya da soruyu uzun paragraflarla genişletmek cevabı daha doğru yapmaz. Örneğin heap sorusunda yalnız karşılaştırmaya giren sayılar istenmiştir; fazladan sayı yazmak hatadır.

Hoca ayrıca bir kodun “ne yaptığı” sorulduğunda cevabın eksiksiz olması gerektiğini söyledi. İkilik tabana çevirme sorusunda kod sayının ikilik karşılığını yalnız “bulmaz”, ekrana da “yazdırır”. “Bulup da ne yapıyor, rafa mı kaldırıyor?” diye sordu.

Heap sorusunda hatalar çok olduğu için hoca, heap konusunun oturmadığını düşünen öğrencilerin o dersleri tekrar izlemesini ve aynı konunun başka anlatımlarına da bakmasını önerdi.

Soru-cevaplar:

- **Soru:** `x->next = x->next->next` satırında atama ters yönde değil mi? Yani "x'in next'inin next'i artık x'in next'ini göstersin" denmiyor mu? **Cevap:** Hayır. Atamada eşittirin solundaki güncellenir. Satır, "x'in next'i, x'in next'inin next'ini göstersin" der. Böylece x'ten sonraki düğüme liste üzerinden erişilemez. O düğümün kendi `next` alanı hâlâ sonraki düğümü gösterir, ama ona artık listeden ulaşılamaz.
- **Soru:** Heap sorusunda yalnız karşılaştırılan sayılar sorulmuştu. İşlemleri yazmasak olur muydu? **Cevap:** Olur. Beklenen yalnız sayılardır. Sayılar küçükten büyüğe (14, 17, 19, 34, 35, 36, 37) ya da karşılaştırma sırasıyla (19, 37, 35, 34, 36, 14, 17) yazılabilir.
- **Soru:** Kod sorusunda süre yetmedi ve kod çalışmadı. Değerlendirme nasıl olacak? **Cevap:** Soruda da önce algoritmanın ve veri yapılarının düzgün tasarlanması, sonra kodun güzelleştirilmesi istenmişti. Hoca kodun genel yapısına ve tasarımına bakacağını söyledi.
- **Soru (aradan sonra):** Dairesel tek bağlı listede başa eleman eklemek $O(n)$ olmaz mı? **Cevap:** Olmaz. Son düğümü gösteren işaretçi (last) baş işaretçisi gibi tutulursa başa ve sona ekleme $O(1)$ olur. Araya eleman ekleme $O(n)$'dir. Hoca "circular linked list insertion" aramasıyla örneklere bakılabileceğini söyledi.

Hoca, sınavın ikinci kısmındaki bağlı liste kod sorusunu derste çözmedi. Daha önce bağlı listeyle kod yazmış olanların bu soruyu rahat çözebileceğini söyledi.

2. Graf Nedir?

Bu haftanın konusu yeni bir veri yapısıdır: graf (graph). Türkçede çizge olarak da adlandırılır. Ancak literatürde genelde "graf" kullanıldığı için derste de bu kelime kullanılacak.

Tanım çok kısadır: **Birbirine bağlı elemanlar kümesine graf denir.** Biraz daha genişletilmiş hâli şudur: Soyut bir matematik nesnesi olarak, birbirine bağlı elemanlardan oluşan kümeye graf adı verilir. Yani graf bir oluşumu, bir birlikteliği ifade eder. Birbiriyle ilişki içinde olan, birbiriyle etkileşen her türlü varlık bir graf oluşturur.

Graflar bilgisayar bilimlerinde sık kullanılır, ama asıl olarak matematiğin ana çalışma alanlarından biridir. Yüzyıllardır üzerinde çalışılır. Bu alana graf teorisi (graph theory) denir.

Hoca tahtaya altı elemanlı bir graf çizdi. Bazı elemanlar arasında birden fazla bağlantı vardı (örneğin A ile B arasında iki farklı ilişki), bir eleman da kendine bağlıydı. A'dan B'ye, B'den C'ye erişilebiliyordu. Bu çizimden grafın iki temel kavramı çıkar:

- **Düğüm (vertex):** Grafın elemanlarıdır. Çoğulu "vertices" olduğu için V ile gösterilir. Düğüm sayısı, küme eleman sayısı gibi $|V|$ ile yazılır. Örnekte $|V| = 6$'dır.
- **Kenar (edge):** Düğümler arasındaki bağlantıyı, ilişkiyi sağlayan parçalardır. E ile gösterilir.

Bir graf $G = (V, E)$ biçiminde yazılır. Bu, düğümler ve kenarlarla tanımlı bir veri grubu demektir.

> **Hocanın vurgusu:** Graf tanımı düğümler ve kenarlar üzerinden yapıldığı için bir graf algoritmasının karmaşıklığı n ile değil, V ve E ile ifade edilir. Örneğin $O(VE)$ yazan bir algoritmanın karmaşıklığı, düğüm sayısı ile kenar sayısının çarpımı mertebesindedir.
>
> **Ders sonrası not:** Hoca $O(VE)$'yi okurken bir yerde "kenar sayısı çarpı düğüm sayısı çarpı kenar sayısı" dedi. Doğrusu yalnız $V \cdot E$'dir.

3. Grafların Kullanım Alanları

Hoca grafların ne kadar yaygın kullanıldığını göstermek için çok sayıda örnek verdi. Amacının, çok alakasız görünen alanlarda bile temelde graf kullanıldığını göstermek olduğunu söyledi.

3.1 İstanbul metro haritası

İstanbul raylı sistem haritasında her durak ya da istasyon bir düğümdür. Raylarla gidilen yollar, yani istasyonlar arasındaki bağlantılar kenardır. Hatların kesiştiği aktarma noktaları da ortak düğümlerdir. Marmaray, Avrupa ve Anadolu yakasını deniz altından bağlayan bir hattır.

Ders sonrası not: Hoca bir yerde "her bir istasyon bir vertex yani bir kenar" dedi. Doğrusu: istasyon düğümdür (vertex), istasyonlar arasındaki ray bağlantısı kenardır (edge).

3.2 İllerin sınır komşulukları ve en kısa yol

Amerikan eyaletlerinin sınır komşulukları standart bir graf örneğidir. Hoca aynı örneği Trakya illeriyle kurdu. Her ilin merkezi bir düğümdür: Edirne, Kırklareli, Tekirdağ, İstanbul ve Çanakkale. Sınır komşusu olan iller arasında kenar vardır:

- Çanakkale - Edirne
- Edirne - Kırklareli
- Edirne - Tekirdağ
- Kırklareli - Tekirdağ
- Kırklareli - İstanbul
- Tekirdağ - İstanbul

Çanakkale ile Tekirdağ arasında kenar yoktur, çünkü hocanın anlatımına göre arada Edirne'ye bağlı Keşan vardır.

Bu sınır komşuluğu grafında "İstanbul'dan Çanakkale'ye gidebilir miyim?" diye sorulabilir. Gidilebilir; örneğin İstanbul → Kırklareli → Edirne → Çanakkale ya da İstanbul → Tekirdağ → Edirne → Çanakkale. Hangisinin seçileceğine karar vermek için kenarlara değer (ağırlık) verilir. Hoca kilometre olarak şu değerleri yazdı:

Kenar	Ağırlık (km)
İstanbul - Kırklareli	150
Kırklareli - Edirne	100
Edirne - Çanakkale	20
Tekirdağ - Edirne	50
İstanbul - Tekirdağ	70
Kırklareli - Tekirdağ	80

İki yolun uzunluğu:

- İstanbul → Kırklareli → Edirne → Çanakkale: $150 + 100 + 20 = 270$ km. Hoca bu yolun Çatalca içinden geçtiğini, çoğunun tek şerit olduğunu da ekledi.
- İstanbul → Tekirdağ → Edirne → Çanakkale: $70 + 50 + 20 = 140$ km. İstanbul-Tekirdağ otoban, Tekirdağ-Edirne duble yoldur.

Bu graf üzerinde bir en kısa yol algoritması çalıştırıldığında en kısa yolun 140 km olduğu bulunur. Kırklareli-Tekirdağ kenarını kullanan yollar daha uzundur (örneğin $150 + 80 + 50 + 20 = 300$ km) (ara adım eklendi).

Ders sonrası not: Kilometre değerleri hocanın örnek için uydurduğu sayılardır. Gerçek mesafeler büyük olasılıkla farklıdır; örneğin Edirne ile Çanakkale merkezleri arası 20 km'den çok daha uzundur.

3.3 Diğer örnekler

- **Protein-protein etkileşim ağları:** Biyoinformatikte sık kullanılır. Her amino asit bir düğüm, aralarındaki etkileşim bir kenardır.
- **Konular arası tıklama bağlantıları:** Bilgisayar bilimleri, matematik, kimya, biyoloji, biyoinformatik gibi konular birbirine bağlıdır. Birine tıklayınca ilişkili diğerine erişilir. Bazı konular merkezdedir ve çok sayıda konuyla ilişkilidir.
- **Facebook arkadaşlıkları:** Siz bir düğümsünüz, arkadaşlarınız da düğüm, arkadaşlık bağı kenardır. Kim kime ne kadar yakın gibi sorular bu graf üzerinde çözülebilir.
- **Haritalar:** Beşiktaş ve Davutpaşa kampüsleri arasında en az mesafeli yolu bulmak en kısa yol problemidir (shortest path). Yolların kapasitesini ve trafiği hesaba katarak en hızlı yolu bulmak ise maksimum akış (max flow) problemidir. Aynı graf üzerinde farklı problemler, farklı algoritmalarla çözülür.

- **Arama motorları:** Her web sayfası ya da kaynak bir düğümdür. Bir sayfadan diğerine bağlantı (link, anchor) varsa bu bir kenardır. Sorgudaki anahtar kelimelerin hangi düğümlerde geçtiği ve bu düğümlerden hangi komşu düğümlere erişilebildiği, arama algoritmalarında kullanılır.
- **Elektronik devreler:** Her devre elemanı bir düğüm, aralarındaki tel ya da lehim bağlantısı bir kenardır. Devrede kısa devre (short circuit) olup olmadığı graflarla analiz edilir. Devre baskılı devre kartına (Printed Circuit Board, PCB) dönüştürülecekse bağlantıların düzlemde çaprazlanmadan çizilip çizilemeyeceği, yani kaç katman gerektiği de graflarla bulunur.
- **Zaman planlama ve tarifeler:** Uçuş ve otobüs tarifeleri, öğretmen-ders atamaları gibi, sırası belirli ve birbirine bağlı işlerin verilen kısıtlar altında en kısa sürede nasıl tamamlanacağı graflarla modellenir.
- **Ticaret ve e-ticaret:** Birliktelik analizlerinde, alıcı-satıcı-ürün ağlarındaki ilişkilerin çıkarılmasında kullanılır.
- **Eşleştirme problemleri:** Her öğrenciye en uygun stajın ya da kulübün bulunması gibi.
- **Bilgisayar ağları:** Her cihaz bir düğümdür. Paketlerin kaynaktan hedefe en hızlı hangi yoldan gideceği (yönlendirme, routing) graf algoritmalarıyla çözülür. Bu, harita örneğindeki Beşiktaş-Davutpaşa problemine benzer.
- **Derleyiciler:** Kodun modülleri ve fonksiyonları arasındaki statik ya da (nesneye yönelik programlamada) dinamik çağrılar ve bunlar için gereken kaynak ayırımı (örneğin bellek tahsisi) graflarla modellenir.
- **Sosyal ağlar:** Olağan dışı bir bağlantının (anomali) tespiti, bir konunun ya da etiketin (hashtag) kim tarafından başlatıldığının, kanaat önderinin (influencer) bulunması, birlikte hareket eden bot ya da trol hesap ağlarının tespit edilmesi graflarla yapılır.

4. Düğüm, Kenar ve Grafın Saklanması

Graf teorisi matematiğin bir alanı olduğu için kendine has bir dili vardır. İlk iki terim düğüm (vertex) ve kenardır (edge).

Düğümler genelde 0'dan $V-1$'e numaralanır. Derste gösterilen örnekte 13 düğüm vardı ve 0'dan 12'ye numaralanmıştı. Bu numaralama, algoritmalarda düğümleri takip etmeyi kolaylaştırır.

Kenarlar düğümleri birbirine bağlar. Örnekte 0 düğümü 1, 2, 5 ve 6 ile bağlıdır. 0'dan dört kenar çıkar; yani 0'ın derecesi (degree) 4'tür. Kenar `v-w` biçiminde yazılır: v düğümü ile w düğümü arasında bir kenar vardır, yani bu iki düğüm birbirine bağlıdır.

> **Ek bilgi (derste anlatılmadı):** Yönsüz bir grafta bir düğümün derecesi, o düğüme değen kenar sayısıdır. Her kenar iki düğümün derecesine birer katkı yaptığı için bütün derecelerin toplamı $2E$'dir. Hoca derece kavramına "birazdan bakacağız" dedi, ama derste ayrıca tanımlamadı.

Bir graf en basit hâliyle bir metin dosyasında şöyle saklanabilir:

1. İlk satıra düğüm sayısı yazılır (örnekte 13).
2. İkinci satıra kenar sayısı yazılır. Örnekte hoca kenarları saydı ve "şansa bak" diyerek 13 kenar olduğunu gördü.
3. Sonra her satıra bir kenar yazılır.

```
13
13
0 1
0 2
0 5
0 6
...
```

Hocanın okuduğu kenarlardan bazıları: `0 1`, `0 2`, `0 5`, `0 6`, `4 6`, `3 5`, `3 4`, `7 8`. Dosyayı okuyan, 13 düğüm (0-12) ve 13 kenar satırı olması gerektiğini bilir. Kenar ağırlıklı bir grafta her satıra ağırlık da eklenebilir.

> **Hocanın vurgusu:** Bir grafın düzlemde nasıl çizildiğinin hiçbir önemi yoktur. Aynı düğümler ve aynı bağlantılar varsa iki farklı çizim aynı grafı gösterir.

Hoca aynı 13 düğümlü grafın iki farklı çizimini gösterdi. İkisinde de 0 düğümü 1, 2, 5 ve 6'ya bağlıdır. İkinci çizimde 9, 10, 11, 12 düğümlerinden oluşan parça başka bir boşluğa taşınmış, 7-8 kenarı dikey yerine yatay çizilmişti. Düğüm sayısı, kenar sayısı ve bağlantılar aynı olduğu için bu iki çizim birebir aynı graftır.

5. Self-Loop, Paralel Kenar, Multigraf ve Basit Graf

Bir düğüm kendine bir kenar içerebilir. Buna self-loop (kendine bağlantı) denir.

İki düğüm arasında birden fazla kenar olabilir. Matematiksel ifadeyle, aynı düğüm çiftini bağlayan birden fazla kenar varsa bunlara paralel kenarlar (parallel edges) denir. Paralel kenar içeren grafa da multigraf (multigraph) denir. Örneğin 0 ile 1 arasında tek değil birden fazla kenar olabilir. Ağırlıksız grafta bunun pek anlamı yoktur. Ama ağırlıklı grafta anlamlıdır: Anadolu ve Avrupa yakası arasında birinci, ikinci ve üçüncü köprü yolları paralel kenarlar gibi düşünülebilir.

Bir grafta hem self-loop hem de paralel kenar yoksa bu grafa basit graf (simple graph) denir. Bu ders kapsamında en çok basit graflar kullanılacak.

> **Ders sonrası not:** Hoca basit grafı tanımlarken bir yerde "çevrim içermeyen" dedi. Burada kastedilen kendine dönen kenardır (self-loop). Basit graf çevrim (cycle) içerebilir; yalnız self-loop ve paralel kenar içermez.

6. Yol, Çevrim ve Bağlantılı Bileşenler

6.1 Yol

Yol (path), kenarlarla birbirine bağlanmış düğümlerin oluşturduğu dizidir; bir silsile, bir zincirdir. Türkçede patika da denir. Eğer yolda tekrar eden düğüm yoksa, yani bütün düğümler farklıysa, buna basit yol (simple path) denir.

Yolun uzunluğu (length), içinden geçilen kenar sayısıdır. Hoca 13 düğümlü grafta aynı iki düğüm arasında üç farklı yol çizdi:

- Birinci yol 4 kenardan geçiyordu, uzunluğu 4'tür.
- İkinci yol 6 kenardan geçiyordu ve yolda bir düğüme tekrar gelinmişti ("kendime bir daha geldim"). Bu yüzden basit yol değildir.
- Üçüncü yol bambaşka bir güzergâhtan 9 kenardan geçiyordu, uzunluğu 9'dur.

Yani aynı kaynak ve hedef arasında farklı uzunlukta birçok yol olabilir.

6.2 Çevrim

Yolun ilk ve son düğümü aynıysa bu yola çevrim (cycle) ya da kapalı yol denir. Başlangıç ve bitiş düğümü dışında tekrar eden düğüm ya da kenar yoksa buna basit çevrim (simple cycle) denir. Çevrimin uzunluğu da içindeki kenar sayısıdır.

6.3 Bağlantılılık ve bileşenler

Grafta verilen iki düğüm arasında bir yol varsa bu iki düğüm bağlıdır (connected). Yol yoksa bağlı değildir (not connected).

Birbirine bağlı düğümlerden oluşan en büyük parçaya bağlantılı bileşen (connected component) denir. Bileşenin içindeki her düğümden diğer her düğüme bir şekilde, bir yol ya da çevrim üzerinden erişilebilir. Hocanın gösterdiği 13 düğümlü grafta üç bağlantılı bileşen vardır. Demek ki bir graf, birbirine bağlı olmayan birden çok bileşenden oluşabilir. Bu gayet doğaldır. Ancak derste genelde tek bileşenli graflarla çalışılacak.

- **Soru:** Graf, birbirine bağlı kümelerin her biri midir, yoksa üç bileşenin hepsi mi bir graftır? **Cevap:** Üç bileşenin tamamı tek bir graftır. Hoca bunu dosya gösterimiyle açıkladı: Toplam düğüm sayısı (örneğin 25), kenar sayısı (örneğin 18) ve kenarlar yazılır. Örneğin 6 ile 12 arasında hiçbir satırda kenar yoksa bu iki düğüm bağlı değildir, ama ikisi de aynı grafın düğümüdür. Yani bir grafta iki düğüm birbirine bağlı olmayabilir.

Hoca bunu Marmaray ile örnekledi. Marmaray yapılmadan önce Anadolu yakasındaki metro hatlarıyla Avrupa yakasındaki hatlar birbirine bağlı değildi. İki ayrı bileşen vardı, ama ikisi birlikte İstanbul metro grafını oluşturuyordu.

7. Graf Türleri: Yönlü, Yönsüz, Ağırlıklı, Ağırlıksız

Şimdiye kadar konuşulan graflar yönsüzdü (undirected). Yönsüz grafta önemli olan yalnız iki düğümün bir kenarla bağlı olmasıdır.

Bağlantının yönü önemliyse bu grafa yönlü graf (directed graph) ya da kısaca digraf (digraph) denir. Kenarlar ok olarak çizilir. Hoca 9, 10, 11, 12 düğümlü bileşene oklar ekledi: 9'dan 10'a, 9'dan 11'e, 9'dan 12'ye gidilebiliyor; 9'dan 12'ye, 12'den de 11'e gidilebiliyor. Ama 10'dan ya da 11'den 9'a gidilemiyor.

Yönsüz grafta 7-8 kenarını bir kez yazmak yeterlidir; ayrıca 8-7 yazmaya gerek yoktur. Yönlü grafta ise yön açıkça belirtilmelidir. Kenar 8'den 7'ye ise 8 7 olarak yazılır ve 8'den 7'ye diye okunur. Yönün "başlangıç, bitiş" mi yoksa "bitiş, başlangıç" mı yazılacağı uygulamaya bağlıdır.

Kenarların değeri de olabilir. Buna kenar ağırlığı (edge weight) denir. Yön ve ağırlık birlikte düşünülünce dört graf türü çıkar:

	Ağırlıksız	Ağırlıklı
Yönsüz	Kenar ağırlıksız yönsüz graf	Kenar ağırlıklı yönsüz graf
Yönlü	Kenar ağırlıksız yönlü graf	Kenar ağırlıklı yönlü graf

Kenar ağırlıklı yönsüz grafta ağırlık bir maliyet, bağlama maliyeti ya da kilometre olarak okunabilir. Örneğin 0-6 kenarının ağırlığı 5 ise 0'dan 6'ya gitmek de, 6'dan 0'a gitmek de 5 km'dir.

Kenar ağırlıklı yönlü grafta gidiş ve dönüş maliyeti farklı olabilir. Hocanın örneğinde 0'dan 1'e 8 km ile gidilirken 1'den 0'a 5 km ile dönülüyordu. Hoca bunu İstanbul trafiğine benzetti: Köprünün bir yönü açıkken öbür yönü hep kapalıdır ve bu durum sabah ile akşam yer değiştirir.

8. Çevrimsiz Graflar ve Ağaçlar

Çevrim içermeyen, yani kapalı döngü içermeyen grafa çevrimsiz graf (acyclic graph) denir.

Hoca, bir çevrimsiz grafın aşağıdaki özellikleri sağlıyorsa bir ağaç (tree) oluşturduğunu söyledi:

- $V - 1$ kenar vardır, yani düğüm sayısından bir eksik kenar.
- Çevrim (döngü) yoktur.
- Bağlantılıdır; ayrı bileşenler yoktur.
- Herhangi bir kenarı kaldırmak yapıyı iki bağımsız bileşene böler.
- Herhangi bir kenar eklemek bir çevrim oluşturur.
- Her düğüm çiftini tek bir basit yol bağlar.

Demek ki ağaçlar grafların özel bir hâlidir. Bu özellikleri sağlayan graflara ağaç denir. Buna bir kısıt daha eklenip, örneğin bir düğümden en fazla iki kenar (iki çocuk) çıkabilir denirse ikili ağaç (binary tree) tanımlanır. Daha genel yapı graftır.

Hoca 19 düğümlü bir örnek gösterdi:

- Bütün düğümler birbirine bağlıdır; tek bir bileşen vardır.
- 19 düğüm ve 18 kenar vardır, yani $V - 1$ kenar.
- Hiç kapalı döngü yoktur.
- Bir kenar kaldırılırsa iki bileşen oluşur. Örneğin bir yaprağa giden kenar kaldırılırsa o düğüm tek başına kalır; daha içteki bir kenar kaldırılırsa bir grup ayrılır.
- Hoca kırmızıyla bir kenar eklediğinde hemen kapalı bir çevrim oluştu.
- Her düğüm çiftini bir basit yol bağlar.

Bu yüzden bu graf özel olarak bir ağaçtır.

Ders sonrası not: Hoca listeyi "beş şart" diye andı, ama altı madde saydı. Ayrıca son maddeyi "her bir kenar çiftini basit bir yol bağlar" diye okudu; doğrusu "her düğüm çiftini tek bir basit yol bağlar" olmalıdır. Bu özelliklerin hepsinin aynı anda kontrol edilmesi gerekmez; bunlar birbirine denktir. Örneğin "bağlantılı ve çevrimsiz" ya da "bağlantılı ve $V-1$ kenarlı" olmak tek başına ağaç olmak için yeterlidir.

9. Graf Yoğunluğu: Seyrek ve Yoğun Graflar

Graflarda bir başka kavram yoğunluktur (graph density). Hoca bunu kabaca, kenarlara bağlı düğümlerin kenar sayısına oranı olarak tarif etti. Bir graf ya seyrektir ya da yoğundur. Karar, düğüm-kenar oranına göre verilir. Hoca literatürde bir katsayı kabul edildiğini, ama buna çok takılmaya gerek olmadığını söyledi.

Daha kullanışlı ayrım şudur:

- **Seyrek graf (sparse graph):** Olabilecek bağlantı sayısına oranla çok daha az kenar içerir.
- **Yoğun graf (dense graph):** Olabilecek bağlantılardan yalnız az sayıda kenarı eksiktir.

Kitaptan alınan örnekte iki graf vardı ve ikisinde de 50 düğüm bulunuyordu. Soldakinde toplam 200 kenar vardı; bazı yerleri sık görünse de seyrek bir graftır. Sağdakinde aynı 50 düğüm arasında 1000 kenar vardı; görselleştirildiğinde karmakarışık görünür ve yoğundur. Seyrek graf metro ağına benzer, yoğun graf ise Facebook'taki arkadaşlık ilişkisine ya da bir protein ağına.

Ders kapsamında en çok seyrek graflarla çalışılacak. Bot ağı ya da kanaat önderi tespiti gibi problemlerde ise yoğun ağlarla çalışılır ve algoritmalar da buna göre farklılaşır.

Ders sonrası not: Hocanın verdiği yoğunluk tanımı ve eşik değeri altyazıda net değildir. Yaygın tanım, yönsüz basit graf için $D = \dfrac{2E}{V(V-1)}$'dir; payda, olabilecek en fazla kenar sayısının iki katıdır. E, V^2 mertebesine yakınsa graf yoğun, V mertebesine yakınsa seyrek kabul edilir. Hocanın örneğinde $V = 50$ için olabilecek en fazla kenar $\binom{50}{2} = 1225$'tir; 200 kenarlı grafın yoğunluğu yaklaşık 0.16, 1000 kenarlı grafın yoğunluğu yaklaşık 0.82'dir.

10. İki Parçalı Graflar

Hoca grafla ilgili son matematiksel tanım olarak iki parçalı grafları (bipartite graph) verdi. Tanımın okunması zor olduğu için iki ayrı tanım yazdığını söyledi:

1. Düğümler, her kenarın iki kümede de birer ucu olacak şekilde iki ayrı kümeye ayrılabiliyorsa graf iki parçalıdır.
2. Daha okunaklı hâli: Düğümleri U ve V diye iki ayrık, birbirinden bağımsız kümeye ayrılabilen ve her kenarı U'daki bir düğümü V'deki bir düğüme bağlayan grafa iki parçalı graf denir.

Slayttaki örnekte bir küme kırmızı, diğer küme siyah düğümlerden oluşuyordu. Her kenar bir kırmızı düğümü bir siyah düğüme bağlıyordu. İki parçalı graflar graf algoritmalarında, özellikle eşleştirme problemlerinde önemlidir.

11. Graf Gösterimleri

Graf bir dosyada düğüm sayısı, kenar sayısı ve kenarlar (varsa ağırlıklarıyla) listelenerek saklanabilir. Asıl soru, grafın veri yapısı olarak bellekte nasıl tutulacağıdır. Hoca dört temel gösterim olduğunu, ama birinden hiç bahsetmeyeceğini söyledi. Bahsedilen üç gösterim kenar listesi, komşuluk listesi ve komşuluk matrisidir. Bunlardan kenar listesi pratikte kullanılmadığı için asıl ikisi üzerinde duruldu.

11.1 Kenar listesi

Kenar listesinde (edge list) bütün kenarlar bir listede tutulur ve her kenar için hangi düğümleri bağladığı saklanır. Bu gösterim pek tercih edilmez. Çünkü en basitinden bir komşuluk bulmak için bütün kenarları gezmek gerekir. Özellikle yoğun grafta hiç kullanışlı değildir. Hoca slaytta bu gösterimin üzerini kırmızıyla çizdi.

11.2 Komşuluk listesi

Komşuluk listesi (adjacency list) özellikle seyrek graflar için çok uygundur ve yer gereksinimini çok azaltır.

Örnek graf yönsüz ve ağırlıksızdır. Hoca bunu kenarların ucunda ok olmamasından (yönsüz) ve kenarlarda değer yazmamasından (ağırlıksız) anladığını söyledi. Beş düğüm vardır. Normalde 0'dan 4'e numaralanması gerekirdi, ama okuması kolay olsun diye 1'den 5'e numaralanmıştır. Çizimin nasıl durduğu önemli değildir; graf ters de çizilebilirdi.

Yapı şöyledir:

- Bir dizi vardır. Dizinin her gözü bir düğüme karşılık gelir. Bu dizi bağlı liste ya da dinamik dizi de olabilir. Düğümde başka bilgi (örneğin isim) tutulacaksa o da bu gözde saklanır.
- Her göz, o düğümün komşularını tutan bir bağlı listenin başını (head) gösterir. Listenin sonu null'dır.

Örnekte 1'den 2'ye ve 5'e gidilebilir, bu yüzden 1 numaralı gözdeki liste $2 \to 5$'tir. Graf yönsüz olduğu için 2'nin listesinde 1 de vardır. Komşuların sırası önemli değildir; genelde gezme sırasına göre küçükten büyüğe yazılabilir.

```
1: 2 -> 5
2: 1 -> 5 -> 3 -> 4
3: 2 -> 4
4: 2 -> 5 -> 3
5: 4 -> 1 -> 2
```

Seyrek grafta kenar sayısı, yani komşu sayısı azdır. Bu yüzden bağlı listeler kısadır ve bir şey aranırken az sayıda elemana bakılır. Toplam karmaşıklık kenar ve düğüm sayısı mertebesindedir.

11.3 Komşuluk matrisi

Komşuluk matrisinde (adjacency matrix) her satır ve her sütun bir düğümü gösterir. Aralarında bağlantı varsa ilgili göze 1, yoksa 0 yazılır. Aynı beş düğümlü yönsüz graf için:

```
   1 2 3 4 5
1  0 1 0 0 1
2  1 0 1 1 1
3  0 1 0 1 0
4  0 1 1 0 1
5  1 1 0 1 0
```

- Köşegen sıfırdır, çünkü hiçbir düğümde self-loop yoktur.
- Graf yönsüz olduğu için matris simetriktir. 1'den 2'ye bağlantı varsa 2'den 1'e de vardır; ikisine de 1 yazılır.
- V düğüm için $V \times V$, yani V^2'lik yer gerekir.

Bir öğrenci, graf yönsüz olduğu için $V^2/2$ yerin yeteceğini söyledi. Hoca bunu kabul etti, ama yönlü grafta yine V^2 yer gerekeceğini ekledi. Bu yüzden genelde komşuluk listesi tercih edilir.

11.4 Yönlü grafta gösterim

Yönlü örnekte altı düğüm vardır. Komşuluklar şöyledir:

- $1 \to 2, 4$
- $2 \to 5$
- $3 \to 5, 6$

- $4 \to 2$
- $5 \to 4$
- $6 \to 6$ (self-loop; 6'nın tek komşusu kendisidir)

Yönsüz grafta 1'den 2'ye bağlantı varsa 2'den 1'e de vardı. Burada 1'den 2'ye bağlantı vardır, ama 2'den 1'e yoktur. Komşuluk matrisi:

```
   1 2 3 4 5 6
1  0 1 0 1 0 0
2  0 0 0 0 1 0
3  0 0 0 0 1 1
4  0 1 0 0 0 0
5  0 0 0 1 0 0
6  0 0 0 0 0 1
```

- Self-loop olduğu için köşegen artık tamamen sıfır değildir: `M[6][6] = 1`.
- Matris simetrik değildir.
- Matris satırdan sütuna okunur: `M[u][v] = 1` ise u'dan v'ye kenar vardır.

Yönlü de olsa yönsüz de olsa iki gösterim de kullanılabilir. Önemli olan, kimin kime bağlı olduğunun bütün düğümler için tutulmasıdır: bağlı listelerin bağlı listesi, dinamik dizide tutulan bağlı listeler ya da daha statik bir matris.

11.5 Hangi gösterim ne zaman avantajlıdır?

Matris kullanmak her zaman kötü değildir. Verilen iki düğüm arasında kenar olup olmadığı sorgulanıyorsa matris çok hızlıdır. Hocanın sade örneği: 3 ile 5 bağlı mı? Matriste yalnız `M[3][5]` gözüne bakılır. Bu $O(1)$'dir. Komşuluk listesinde ise 3'ün bütün komşularına, null görene kadar tek tek bakmak gerekir. Bu $O(\deg(3))$'tür, yani düğümün derecesine bağlıdır.

> **Ders sonrası not:** Hoca önce 1 ile 5 arasındaki bağlantıyı yönlü örnek üzerinden anlatmaya başladı ve "5'e kimler geliyor, 2'ye kimler geliyor" diye bir yol takip etti. Kenar sorgusu için yol takip etmek gerekmez. Komşuluk listesinde u-v kenarı, yalnız u'nun listesi taranarak bulunur; bu $O(\deg(u))$ sürer. Hoca da ardından daha sade 3-5 örneğine geçti.

Bir düğümün bütün komşularını gezmek gerekiyorsa durum tersine döner. Matriste bütün satır, yani V göz taranır. Listede ise yalnız komşu sayısı kadar gezilir. "Verilen bir düğümün bütün komşuları için bir şey yap" türündeki işlemler graf algoritmalarında çok sık olduğu için komşuluk listesi genelde daha düşük karmaşıklıklı çözümler verir.

Hoca slayttaki karşılaştırma tablosunu şöyle açıkladı:

İşlem	Kenar listesi	Komşuluk matrisi	Komşuluk listesi
Bellek	E	V^2	$E+V$
Kenar ekleme	1	1	1
`v-w` kenarı var mı?	E	1	$\deg(v)$
v'nin tüm komşularını gezme	E	V	$\deg(v)$

- **Kenar listesi:** Kenar sayısı kadar yer kaplar. Kenar eklemek ucuzdur; listeye yeni kenar eklenir. Ama komşuluk sorgusu ve komşuları gezmek için bütün kenarlar gezilmelidir.
- **Komşuluk matrisi:** V^2 yer kaplar. Satır ve sütun sayısı sabittir. Kenar eklemek sabit zamanlıdır; örneğin 2'den 3'e yeni bağlantı kurulursa ilgili göze 1 yazılır.
- **Komşuluk listesi:** Kenar sayısı artı düğüm sayısı kadar düğüm (node) gerektirir. Kenar eklemek sabit zamanlıdır; yeni komşu, ilgili gözdeki listenin başına eklenir.

  Ek bilgi (derste anlatılmadı): Yönsüz grafın komşuluk listesinde her kenar iki kez görünür: `v-w` kenarı hem v'nin hem w'nin listesindedir. Bu yüzden listelerdeki toplam eleman sayısı $2E$'dir. Bellek yine $O(V+E)$ mertebesinde kalır. Yukarıdaki beş düğümlü örnekte 7 kenar vardır ve listelerde toplam 14 eleman bulunur.

- **Soru:** Komşuluk listesini matrise tercih etmekte grafın yönlü ya da yönsüz olmasının bir etkisi var mı? Örneğin yönlüde matris, yönsüzde liste daha mı iyi? **Cevap:** Hayır, bir önemi

yoktur. Gerçek hayat problemlerinde genelde yönlü graflarla çalışılır; yalnız bağlantı var mı yok mu sorusunda yönsüz graf kullanılır. Yönlü bağlantılar daha az sayıda olduğu için komşu sayısı daha da azalır ve komşuluk listesi daha da avantajlı hâle gelir.

Hocanın vurgusu: Bir düğümün komşu sayısıyla çalışmak, toplam düğüm sayısıyla çalışmaktan her zaman çok daha avantajlıdır. Facebook'ta toplam düğüm sayısı milyarlar mertebesindedir, ama bir kişinin komşu (arkadaş) sayısı en fazla birkaç bin mertebesindedir. Bir algoritmada milyarlarla değil binlerle çalışmak tercih edilir. Bu yüzden genelde komşuluk listesi kullanılır.

12. Asgari Tarama Ağacı Problemi

Ağaçların grafların özel bir hâli olduğu söylenmişti. Şimdi verilen bir graf için özel bir ağaç tanımlanır: asgari tarama ağacı (minimum spanning tree, MST). Türkçede asgari örten ağaç, minimum tarayan ağaç, minimum örten ağaç gibi adlar da kullanılır. İngilizcedeki "span" fiili yayılmak, örtmek, kaplamak anlamına gelir.

12.1 Muhtar problemi

Hoca problemi bir köy örneğiyle anlattı. Bir köyün muhtarısınız. Sorumluluğunuz köydeki bütün evleri birbirine bağlamak; bir belediye gibi yol ya da elektrik getirmeniz gerekiyor. Kurallar:

- Bütün evler birbiriyle bağlı olacak. Bir evden çıkan, bir şekilde köydeki bütün evlere erişebilmeli.
- Bir yol parçası yalnız ve yalnız iki evi birbirine bağlayabilir. Yolu ortadan dallandırıp üçüncü bir eve bağlamak olmaz. Yani bağlantılar mutlaka verilen düğümler arasında kurulur.
- Bütçe kısıtlıdır ve her yolun bir maliyeti (asfaltlama, bakım, tamir masrafı) vardır. Yani her kenarın bir ağırlığı vardır.

Hoca basit olsun diye dört evli bir örnek çizdi. Evlere Ayşe, Banu, Cemre ve Defne adlarını verdi. Muhtar önce her evi diğer bütün evlere bağladı: Ayşe-Banu, Ayşe-Cemre, Ayşe-Defne, Banu-Defne, Banu-Cemre ve Cemre-Defne. Hoca yollara 3, 5, 1, 8, 2 ve 4 birimlik maliyetler yazdı. Toplam harcama:

$$8 + 3 = 11, \quad 11 + 1 = 12, \quad 12 + 2 = 14, \quad 14 + 5 = 19, \quad 19 + 4 = 23$$

Bütün yollar yapılınca herkes mutlu olur, ama muhtar 23 birim harcamıştır. Hedef ise bütün evleri bağlayan ve toplam masrafı en az olan yol ağını kurmaktır.

Hoca en ucuz yolları seçerek çözdü. Önce maliyeti 1 olan yolu yaptı, sonra 3'lük yolu ekledi; toplam 4 birimle üç ev bağlandı. Sonra 2'lik yolu ekledi. Seçilen üç yol da Banu'nun evine bağlanıyordu:

- Ayşe, Banu üzerinden Cemre'ye ve Defne'ye gidebilir.
- Cemre, Banu üzerinden Ayşe'ye ve Defne'ye gidebilir.
- Defne, Banu üzerinden Ayşe'ye ve Cemre'ye gidebilir.

Hoca "en şanslısı Banu çıktı" dedi. Dört ev yalnız $3 + 1 + 2 = 6$ birim masrafla birbirine bağlandı. Hoca bu muhtara "tutumlu muhtar" dedi. Seçilen üç yol en ucuz üç yol olduğu ve bir çevrim oluşturmadığı için bu, ulaşılabilecek en küçük toplamdır (ara adım eklendi).

Dört evde ve 1, 2, 8 gibi belirgin sayılarla bunu bakarak bulmak kolaydır. Ama köyde 100 ev varsa ve hepsinin arasındaki yol maliyetleri biliniyorsa, hangi kombinasyonun en ucuz yol ağını vereceği nasıl bulunur? Bu problemin çözümü asgari tarama ağacından geçer.

12.2 Problemin tanımı

- Köy yönsüz bir graf olarak modellenir: $G = (V, E)$. Yön yoktur, çünkü önemli olan evlerin birbirine bağlı olup olmamasıdır.
- Her $(u, v) \in E$ kenarı için bir ağırlık $w(u, v)$ atanır.

- Öyle bir $T \subseteq E$ alt kümesi bulunur ki T bütün düğümleri birbirine bağlasın (tarasın, örtsün) ve toplam ağırlık en küçük olsun:

$$w(T) = \sum_{(u,v)\in T} w(u,v)$$

En kötü durumda T, E'nin kendisine eşit olabilir. Bütün tarama ağaçları (spanning tree) içinde ağırlıkları toplamı en az olan ağaca, yani muhtarın en az para harcadığı yol ağına, asgari tarama ağacı denir.

> **Ek bilgi (derste anlatılmadı):** Bir grafın tarama ağacı, grafın bütün düğümlerini içeren ve ağaç olan alt grafıdır. Ağaç olduğu için $V - 1$ kenar içerir ve çevrim içermez. Bağlantılı her grafın en az bir tarama ağacı vardır. Kenar ağırlıklarının hepsi farklıysa asgari tarama ağacı tektir; eşit ağırlıklar varsa birden fazla olabilir.

12.3 Slayttaki örnek

Slayttaki köyde 9 ev (düğüm) ve yapılabilecek 14 yol adayı (kenar) vardır. Hoca bazı kenarları okudu: A ile B bağlanabilir ve 11'e mal olur; B ile D bağlanabilir ve 8'e mal olur. B ile E doğrudan bağlanamaz, çünkü arada bir göl ya da dağ vardır. Aynı şekilde D ile F arasında da başka bir su birikintisi olduğu için bağlantı kurulamaz. Coğrafya böyle olduğu için bu düğüm çiftleri arasında kenar yoktur.

Kenar listesinin tamamı altyazıda okunmuyor. Aşağıdaki tablo, aynı dersin kısa notundaki slayt dökümünden alınmıştır:

Kenar	Ağırlık	Kenar	Ağırlık
a-b	10	d-e	7
a-c	12	d-g	8
b-c	9	d-h	5
b-d	8	f-h	6
c-e	3	g-h	9
c-f	1	g-i	2
e-f	3	h-i	11

9 düğümü bağlamak için $V - 1 = 9 - 1 = 8$ kenar seçmek gerekir. Öyle "sihirli" 8 kenar seçilmelidir ki 9 düğümün hepsi birbirine bağlansın ve toplam maliyet en az olsun.

Kaba kuvvetle düşünülürse 14 kenardan 8'inin seçilmesi $\binom{14}{8}$ olasılık demektir. Bu kadar olası alt küme içinden bir ya da birkaçı en küçük toplam ağırlığa sahiptir. Hangisinin olduğunu bulmak için daha akıllı yöntemler gerekir.

Kısa nottaki MST kenarları: c-f (1), g-i (2), e-f (3), d-h (5), f-h (6), b-d (8), d-g (8), a-b (10). Toplam ağırlık 43'tür.

> **Ders sonrası not:** Tablodaki kenarlarla bütün 8 kenarlı alt kümeler python ile denendi. En küçük toplam 43'tür ve bu toplamı veren iki farklı asgari tarama ağacı vardır: biri e-f (3), diğeri c-e (3) kenarını içerir. Hocanın söylediği gibi a-b kenarı 11 ise asgari toplam 44 olur; a-b her iki durumda da ağaca girer.

12.4 Sonraki ders

Hoca konuyu burada durdurdu. Asgari tarama ağacının nasıl bulunacağı bir sonraki derste işlenecek: önce genel (jenerik) bir yol düşünülecek, ardından daha etkin çözümlere ve farklı çözüm yaklaşımlarına bakılacak.

Örnek Sorular

Aşağıdaki sorular, vizenin birinci kısmında sorulup derste çözülen sorulardır. Sıra, hocanın derste izlediği sıradır. Son soru derste sözlü sorulmuştur.

Soru 1 (derste çözüldü, 00:03:29)

Aşağıdaki yarı kod $n = 20$ için ekrana ne yazar? Kod ne iş yapar?

```
while n > 0:
    push(stack, n mod 2)
    n = n / 2
while stack boş değil:
    print(pop(stack))
```

Çözüm: Döngü adım adım:

n	n mod 2 (push)	$n/2$
20	0	10
10	0	5
5	1	2
2	0	1
1	1	0

Yığına sırayla 0, 0, 1, 0, 1 girer. $n = 0$ olunca ilk döngü biter. Yığın boşaltılırken son giren ilk çıkar: ekrana `10100` yazılır. Kod, verilen sayının ikilik tabandaki karşılığını bulur **ve ekrana yazdırır**.

Yarı kodun C karşılığı (nota eklendi, `gcc -Wall` ile derlendi, çıktı `10100`):

```c
#include <stdio.h>

#define MAX 32

int stack[MAX];
int top = -1;

void push(int x) { stack[++top] = x; }
int pop(void) { return stack[top--]; }
int isEmpty(void) { return top == -1; }

int main(void) {
    int n = 20;
    while (n > 0) {
        push(n % 2);
        n = n / 2;
    }
    while (!isEmpty())
        printf("%d", pop());
    printf("\n");
    return 0;
}
```

Soru 2 (derste çözüldü, 00:06:58)

Bir kuyrukta başlangıçta sırasıyla 5, 2, 1, 4 sayıları vardır. Kuyruk boş olmadığı sürece dequeue edilen eleman yığına push edilir. Sonra yığın boş olmadığı sürece pop edilen eleman kuyruğa enqueue edilir. Son olarak kuyruk boş olmadığı sürece dequeue edilen eleman yazdırılır. Ekrana ne yazılır?

Çözüm: Kuyruk ilk giren ilk çıkar (FIFO), yığın ilk giren son çıkar (LIFO) çalışır.

1. Kuyruktan sırayla 5, 2, 1, 4 çıkar ve yığına itilir. Yığının tepesinde 4 vardır.
2. Yığından sırayla 4, 1, 2, 5 çıkar ve kuyruğa eklenir. Kuyruk: 4, 1, 2, 5 (başı 4).
3. Kuyruktan sırayla çıkarılıp yazdırılır: **4 1 2 5**.

Derste en sık yapılan hata, cevabı 5, 2, 1, 4 olarak yazmaktı.

Soru 3 (derste çözüldü, 00:09:40)

Bir bağlı listede son düğüm olmadığı bilinen bir `x` düğümü için `x->next = x->next->next;` satırı ne yapar?

Çözüm: "Son düğüm olmadığı bilinen" ifadesi, `x->next->next` erişiminde hata oluşmasın diye konmuştur. Satır, `x`'in `next` alanını bir sonraki düğümün sonrasına bağlar. Böylece **x'ten sonra gelen düğüm listeden çıkarılır**. `x` silinmez. Çıkarılan düğüm bellekten de silinmez; yalnız liste üzerinden ona erişilemez. Derste en sık yapılan hata "x'i siler" demekti.

> **Ders sonrası not:** C'de çıkarılan düğüm bellekten serbest bırakılmazsa bellek sızıntısı oluşur. Bunu önlemek için düğüm önce geçici bir işaretçide tutulur, bağlantı güncellenir, sonra `free` edilir. Hoca, bazı öğrencilerin cevaba bunu eklediğini, ama sorunun yalnız satırın ne yaptığını sorduğunu söyledi. (C ile denendi: 1 → 2 → 3 listesinde `x` ilk düğümken satır çalışınca liste 1 → 3 olur.)

Soru 4 (derste çözüldü, 00:14:18)

Bir heap ağacı veriliyor. En büyük elemanın silinip ağacın yeniden heapify edilmesi sırasında hangi sayılar karşılaştırma işlemine girer?

Çözüm: Önce ağacın max heap olduğu kontrol edilir: her düğüm çocuklarından büyüktür.

1. Kök 38 silinir. Son eleman 19 köke yazılır ve son düğümün bağı koparılır (heap boyutu bir azalır). Bu adımda karşılaştırma yoktur.
2. 19, çocukları 37 ve 35 ile karşılaştırılır. En büyüğü 37'dir; 37 yukarı çıkar, 19 aşağı iner.
3. 19, yeni çocukları 34 ve 36 ile karşılaştırılır. En büyüğü 36'dır; 36 yukarı çıkar, 19 aşağı iner.
4. 19, yeni çocukları 14 ve 17 ile karşılaştırılır. 19 ikisinden de büyüktür; yer değişmez ve işlem biter. Daha alttakilerle karşılaştırma gerekmez.

Karşılaştırmaya giren sayılar: **19, 37, 35, 34, 36, 14, 17** (7 sayı). Küçükten büyüğe: 14, 17, 19, 34, 35, 36, 37. Yanlış tarafa gidip 25 ya da 23 ile, alakasız 15 ya da 16 ile veya ağacın geri kalanının hepsiyle karşılaştırmak hatadır.

Soru 5 (derste çözüldü, 00:19:22)

Özyinelemeli (recursive) bir fonksiyonda temel durum (base condition) yoksa ne olur? Şıklar: fonksiyon çalışmaz; tüm koşullar sağlanana kadar devam eder; derleme hatası verir; probleme bağlı olarak base condition tanımlanmayabilir; sonsuz çalışma durumu oluşur.

Çözüm: Şıklar tek tek elenir:

- "Fonksiyon çalışmaz" yanlıştır; fonksiyon çalışır.
- "Tüm koşullar sağlanana kadar devam eder" yanlıştır; temel durum olmadığı için sağlanacak bir koşul yoktur.
- "Derleme hatası verir" yanlıştır; derleme zamanında bir hata yoktur, sorun çalışma zamanındadır.
- "Probleme bağlı olarak base condition tanımlanmayabilir" yanlıştır; özyinelemenin durması için temel durum mutlaka tanımlanmalıdır.

Doğru cevap D şıkkıdır: **sonsuz çalışma durumu oluşur.** Teoride sonsuza kadar sürer. Pratikte fonksiyon çağrı yığını (call stack) şişer ve yığın taşması (stack overflow) ile program durur.

Soru 6 (derste çözüldü, 00:21:42)

Tek bağlı liste (singly linked list) ile gerçekleştirilen bir kuyrukta ilk eklenen elemanın adresi saklanıyor, son eklenen eleman için ayrıca adres saklanmıyor. `enqueue` ve `dequeue` işlemlerinin en kötü durum zaman karmaşıklığı nedir?

Çözüm:

- **`enqueue`: $O(n)$.** Elde yalnız ilk elemanı gösteren işaretçi (head, first) vardır. Yeni eleman eklemek için `next` alanı null olan son düğüme kadar `next, next, next` diye gidilir. Yeni

düğüm oluşturulup son düğümün `next`'i ona bağlanır, yeni düğümün `next`'i null yapılır. n eleman geçilmek zorundadır.
- **dequeue: $O(1)$.** İlk elemanın değeri geçici değişkene alınır, baş işaretçisi `next`'e güncellenir, istenirse eski düğüm `free` edilir. Sabit sayıda işlem yapılır.

En sık yapılan hata ikisini ters yazmaktı.

Soru 7 (derste çözüldü, 00:25:07)

Tamamı aynı sayıdan oluşan n elemanlı bir dizi (örneğin bütün elemanlar 3) selection sort ve insertion sort ile sıralanırsa karmaşıklıklar ne olur?

Çözüm:

- **Selection sort: $O(n^2)$.** Her adımda kalan elemanların içindeki minimumu (ya da maksimumu) bulur ve ilgili gözle yer değiştirir. Sonra boyutu bir azaltıp devam eder. İç içe iki döngü vardır; yaklaşık $n^2/2$ karşılaştırma yapar. Dizinin durumuna bakmaz, en iyi durumda da en kötü durumda da "körlemesine" n^2 mertebesinde çalışır.
- **Insertion sort: $O(n)$.** Her eleman, solundaki elemanla bir kez karşılaştırılır. Elemanlar eşit olduğu için hiç yer değiştirme olmaz. Toplam $n-1$ karşılaştırma yapılır; karmaşıklık doğrusaldır.

(Python ile $n = 10$ için sayıldı: insertion sort 9, selection sort 45 karşılaştırma yaptı.) Hatalı cevaplarda çoğunlukla "tamamı aynı sayıdan oluşan" ifadesi gözden kaçmış ve ikisine de n^2 yazılmıştı.

Soru 8 (derste çözüldü, 00:28:01 ve 01:01:41)

Dairesel bağlı listenin (circular linked list) önüne yeni bir eleman eklemenin en kötü durum karmaşıklığı nedir?

Çözüm: Dairesel listede son düğüm null'ı değil, ilk düğümü gösterir. Önüne eleman eklemek için yeni düğüm oluşturulur ve bağlantıları kurulur; sabit sayıda işlem yeterlidir: $O(1)$. Aradan sonra gelen soruya hoca şöyle cevap verdi: Son düğümü gösteren işaretçi (last) baş işaretçisi gibi tutulursa hem başa hem sona ekleme $O(1)$'dir; araya ekleme $O(n)$'dir. Bu durumda yeni düğümün `next`'i `last->next`'e (ilk düğüme), `last->next` de yeni düğüme bağlanır (ara adım eklendi).

Soru 9 (derste çözüldü, 00:29:38)

Bir ikili arama ağacına (BST) en kötü durumda eleman eklemenin karmaşıklığı nedir?

Çözüm: BST'nin en kötü durumu bağlı liste hâlidir. Örneğin elemanlar 2, 4, 6, 8 sırasıyla eklenirse her biri bir öncekinin sağına gelir ve ağaç tek dallı bir zincire dönüşür. Yeni eleman eklemek için bütün zincir gezilir: $O(n)$. Ortalama durumda karmaşıklık $O(\log n)$'dir, ama soru özellikle en kötü durumu sormaktadır.

Soru 10 (derste çözüldü, 00:30:50)

Aşağıdaki yarı kodda `random()` 0 ya da 1 döndürür. `1 3 5 2 4` çıktısı oluşabilir mi? Neden?

```
for i = 1 to 5:
    if random() == 1:
        print(i)
    else:
        push(stack, i)
while stack boş değil:
    print(pop(stack))
```

Çözüm: `i` 1'den 5'e artarak gider. 1 gelirse `i` hemen yazdırılır, 0 gelirse yığına itilir. `1 3 5` kısmı oluşabilir: 1, 3 ve 5 için 1, 2 ve 4 için 0 gelmiştir. Bu durumda 2 ve 4 yığına bu sırayla, yani küçükten büyüğe itilmiştir. Döngüden sonra yığın boşaltılırken son giren ilk çıkar; bu

sayılar ancak 4 2 sırasıyla yazdırılabilir. Yığından küçükten büyüğe sıralı bir çıktı alınamaz. Bu yüzden **1 3 5 2 4 oluşamaz.** Sayıların rastgele seçilmesi bunu değiştirmez.

Soru 11 (derste çözüldü, 00:35:14)

Aynı yarı kodda 1 3 5 4 2 çıktısı oluşabilir mi?

Çözüm: Oluşabilir. 1, 3 ve 5 için 1 gelirse bunlar doğrudan yazdırılır. 2 ve 4 yığına itilir ve yığın boşaltılırken 4, 2 sırasıyla yazdırılır. (Python ile 32 olası rastgele dizinin hepsi denendi: 1 3 5 4 2 çıktılar arasında var, 1 3 5 2 4 yok.)

Hoca, bazı öğrencilerin bu iki soruyu aynı soru sandığını, ama birinde 2 4, diğerinde 4 2 olduğunu hatırlattı.

Soru 12 (derste çözüldü, 00:36:30)

Sırasız bir A_n dizisi ve sıralı bir B_m dizisi veriliyor; $n \gg m$. İki dizinin ortak elemanlarını bulan bir algoritma tasarlayınız ve karmaşıklığını veriniz.

Çözüm: Burada $\gg$ "çok büyüktür" anlamındadır; C'deki kaydırma (shift) operatörü değildir. Ortak eleman bulmanın anahtarı aramadır: bir dizinin elemanları diğerinde aranır. B sıralı olduğu için ikili arama (binary search) kullanılabilir. m elemanlı dizide ikili arama $O(\log m)$ sürer. A'daki n elemanın her biri için bir kez yapılır:

$$O(n \log m)$$

İki iç içe döngüyle doğrusal arama $O(nm)$ sürer ve verimsizdir. "m küçük olduğu için ihmal edilir, $O(n)$ olur" demek de yanlıştır. Soru açıkça "etkin algoritma" dememiştir, ama bir tasarım isteniyorsa algoritma tasarım ilkelerine uygun ve etkin olması beklenir.

Soru 13 (derste çözüldü, 00:39:31)

Sayı değerleri harflerle maskelenmiş bir ikili arama ağacı veriliyor (A-M, 13 düğüm).

a) Sıralamada A düğümünden büyük ilk değer hangisidir?

b) B düğümünden büyük ilk değer hangisidir?

c) C düğümünden büyük ilk değer hangisidir?

d) Ağacın yüksekliği kaçtır?

e) Ağacın mevcut yüksekliğini bozmadan kaç eleman daha eklenebilir?

Çözüm: Bir düğümden büyük ilk değer (sıralamada onu izleyen değer), sağ alt ağacın en soldaki düğümüdür. Sağ çocuğun kendisi değildir.

- a) A'nın sağ çocuğu C, A'dan büyüktür ama ilk büyük değer değildir. C'den küçük F ve F'den küçük J de A'dan büyüktür. Sağ alt ağaçta en sola gidilir: **J**. Derste en çok düşülen tuzak C yazmaktı.
- b) B'nin sağına gidilir, sonra en sola gidilmeye çalışılır. Ama E'nin solu boştur. Bu yüzden cevap **E**'dir.
- c) C'nin sağ çocuğu G'dir, ama sağ alt ağacın en küçük elemanı en soldaki **K**'dir. G yazmak aynı tuzağa düşmektir.
- d) Kökten en dipteki çocuğa (M) 4 kenar vardır: yükseklik **4**.
- e) Yükseklik 4 olan bir ikili ağaçta 5 seviye vardır. Hocanın sayımı: 3. seviyeyi doldurmak için 3 boşluk, son seviyede 16 yerden yalnız 1'i dolu olduğu için 15 boşluk. Toplam $3 + 15 = 18$. Aynı sonuç kapasiteden de çıkar: yükseklik 4 olan ikili ağaç en fazla $2^5 - 1 = 31$ düğüm taşır; ağaçta 13 düğüm vardır; $31 - 13 = 18$ (ara adım eklendi). Cevap **18**.

Soru 14 (derste sözlü, 02:22:45)

9 düğümlü ve 14 kenarlı grafta asgari tarama ağacı için 8 kenar seçilecekse kaç farklı kenar alt kümesi vardır?

Çözüm: Hoca "14'ün 8'li kombinasyonu kaç yapıyor? Hesaplarsınız" dedi. Derste bir öğrencinin "3.300" gibi bir sayı söylediği duyuluyor.

> **Çözüm (derste verilmedi):** $\binom{14}{8} = \dfrac{14!}{8!\,6!} = 3003$. Python ile doğrulandı. Bu alt kümelerin hepsi tarama ağacı değildir; bazıları çevrim içerir ve bazı düğümleri dışarıda bırakır.

Dersin Toparlaması

Dersin ilk bölümünde vize sorularının üzerinden geçildi. Sorular yığın, kuyruk, bağlı liste, heap, özyineleme, sıralama algoritmaları ve ikili arama ağacı konularını kapsıyordu. Hocanın ortak mesajı, sorulardaki "en kötü durum", "tamamı aynı sayıdan oluşan", "ilk büyük değer" gibi ayrıntıların dikkatle okunması ve yalnız sorulanın cevaplanmasıydı.

İkinci bölümde yeni bir veri yapısı olan graf başladı. Graf, düğümler ve kenarlardan oluşan $G = (V, E)$ yapısıdır ve karmaşıklıklar V ile E cinsinden yazılır. Self-loop, paralel kenar, basit graf, yol, çevrim, bağlantılı bileşen kavramları; yönlü/yönsüz ve ağırlıklı/ağırlıksız graf türleri; çevrimsiz graf ile ağaç arasındaki ilişki; seyrek ve yoğun graflar ile iki parçalı graflar tanımlandı. Grafın bellekte gösterimi için kenar listesi, komşuluk listesi ve komşuluk matrisi karşılaştırıldı; gerçek problemlerde genelde komşuluk listesi tercih edilir. Ders, muhtarın köydeki evleri en az masrafla bağlama problemiyle tanımlanan asgari tarama ağacı (MST) ile bitti. MST'yi bulan algoritmalar bir sonraki derste işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.